Министерство науки и высшего образования Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

“НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО”

Дисциплина Тестирование программного обеспечения

Лабораторная работа №1

Вариант: 368273

Выполнили:
Карандашева Анастасия Денисовна
Ястребов-Амирханов Алекси
Группа: P3332

Преподаватель:
Чупанов Аликылыч Алибекович

г. Санкт-Петербург

2026

Содержание

Задание 3
Часть 1 3
Точки для тестирования 4
Таблица ожидаемых значений 5
Тесты разложения cos(x) 5
Результаты тестирования 7
Часть 2 9
Планируемые к покрытию операции и свойства 10
1. Поиск минимального элемента (min / getMinimum) ..... **Ошибка! Закладка не определена.**
2. Добавление элемента (insert) ..... **Ошибка! Закладка не определена.**
3. Удаление минимального элемента (removeMin) ..... **Ошибка! Закладка не определена.**
4. Объединение куч (merge) ..... **Ошибка! Закладка не определена.**
5. Уменьшение ключа (decreaseKey) ..... **Ошибка! Закладка не определена.**
Структура тестов ..... **Ошибка! Закладка не определена.**
Тесты для фибоначчиевой кучи 11
Результаты тестирования 16
Часть 3 17
Описание классов и их методов 17
Структура тестов ..... **Ошибка! Закладка не определена.**
Тесты доменной модели 18
Результаты тестирования 19
Выводы 19

Задание

1. Для указанной функции провести модульное тестирование разложения функции в степенной ряд. Выбрать достаточное тестовое покрытие.

2. Провести модульное тестирование указанного алгоритма. Для этого выбрать характерные точки внутри алгоритма, и для предложенных самостоятельно наборов исходных данных записать последовательность попадания в характерные точки. Сравнить последовательность попадания с эталонной.

3. Сформировать доменную модель для заданного текста. Разработать тестовое покрытие для данной доменной модели.

Вариант 368273:

1. Функция cos(x)
2. Программный модуль для работы с Фибоначчиевой кучей (Internal Representation, http://www.cs.usfca.edu/~galles/visualization/FibonacciHeap.html)
3. Описание предметной области:
   После недолгой поездки на аэромобиле Артур и старый магратеянин оказались у какой-то двери. Они вышли из машины и прошли в приемную, уставленную стеклянными столиками и наградами из прозрачного пластика. Почти сразу же над дверью в другом конце комнаты загорелся свет, и они вошли в нее.

Часть 1

Реализация разложения функции cos(x):
ITMO/ТПО/Лаб1/Code/TPOLAB1/src/main/java/org/example/math/Cosine.java at master · AlexandroLe/ITMO

В коде используется разложение функции косинуса в ряд Тейлора (ряд Маклорена) в точке 0:

$$\cos x = \sum_{k=0}^{\infty} (-1)^k \frac{x^{2k}}{(2k)!}$$

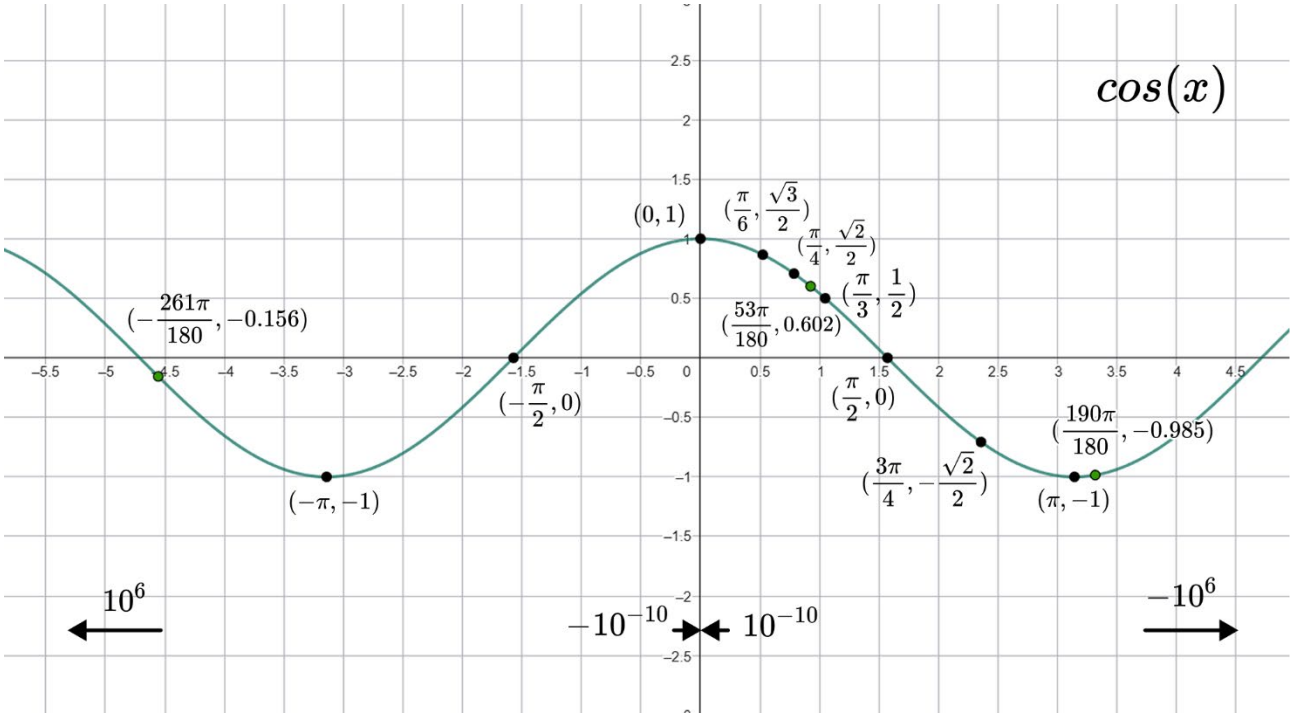
Первые члены ряда:

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \frac{x^8}{8!} - \dots$$

В реализации используется рекуррентная формула для вычисления следующего члена:

$$a_k = a_{k-1} \cdot \frac{-x^2}{(2k-1)(2k)}$$

Точки для тестирования



0 – центр разложения ряда Маклорена; значение должно быть **ровно 1**, проверяется корректность работы с точкой разложения и минимальная погрешность.

±10⁻¹⁰ – очень маленькие числа, близкие к нулю; проверяется, что функция корректно сохраняет чётность и не округляется преждевременно к 1.

π/6, π/4, π/3 – дробные углы в первой четверти; проверяется точность редукции аргумента и корректность вычисления значений косинуса, близких к √3/2, √2/2, 1/2.

3π/4, 53π/180 – углы во второй четверти; проверяется правильность смены знака, корректная редукция к диапазону [-π/2, π/2], точность при произвольных углах.

π/2, -π/2 – границы первой и второй четверти; значение должно быть близко к **0**, проверяется обработка ветвления аргумента.

π, -π – границы полу-периода; проверяется корректность редукции по модулю 2π и возвращение значения **-1**.

190π/180, -261π/180 – углы за пределами [-π, π]; проверяется корректность редукции аргумента и вычисления косинуса для углов, выходящих за один период.

±10⁶ – большие по модулю значения; проверяется стабильность редукции по 2π и отсутствие численных ошибок при больших аргументах.

Примечания по тестированию

- Для больших аргументов (|x| > 1000) допускается погрешность **1e-9**, для остальных – **1e-12**.

- Проверяется, что использовано **>0 членов ряда** и **не превышено максимально допустимое количество членов** (<=1000), чтобы убедиться, что вычисление действительно происходит, а не возвращается константа.
- Все значения заданы через BigDecimal, чтобы избежать проблем с переполнением.

Таблица ожидаемых значений

x	**cos(x)**
0	1.0
10^{-10}	0.9999999999999999
-10^{-10}	0.9999999999999999
π/6 ≈ 0.5235987755982988	0.8660254037844387
π/4 ≈ 0.7853981633974483	0.7071067811865476
π/3 ≈ 1.0471975511965976	0.5000000000000001
3π/4 ≈ 2.356194490192345	-0.7071067811865475
53π/180 ≈ 0.9238398270311391	0.6018150231520482
π/2 ≈ 1.5707963267948966	6.123233995736766e-17 ≈ 0.0
-π/2 ≈ -1.5707963267948966	-6.123233995736766e-17 ≈ 0.0
π ≈ 3.141592653589793	-1.0
-π ≈ -3.141592653589793	-1.0
190π/180 ≈ 3.3161255787892263	-0.9848077530122081
-261π/180 ≈ -4.547001614823535	-0.15643446504023092
10^{6}	0.9367521275331447
-10^{6}	0.9367521275331447

Тесты разложения cos(x)

```
package org.example.math;

import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.MethodSource;
import org.junit.jupiter.params.provider.Arguments;

import java.math.BigDecimal;
import java.math.MathContext;
import java.util.stream.Stream;
```

```java
import static org.junit.jupiter.api.Assertions.*;

class CosineParameterizedTest {

    private static final MathContext MC = new MathContext(50);

    @ParameterizedTest(name = "[{index}] x={0}")
    @MethodSource("valuesProvider")
    void parameterizedCosTests(BigDecimal x) {

        BigDecimal actual = Cosine.cos(x);

        double expectedDouble = Math.cos(x.doubleValue());
        BigDecimal expected = BigDecimal.valueOf(expectedDouble);

        BigDecimal tolerance = x.abs().compareTo(new BigDecimal("1000")) > 0
                ? new BigDecimal("1e-9")  // для больших аргументов
                : new BigDecimal("1e-12"); // для обычных

        BigDecimal diff = actual.subtract(expected, MC).abs();

        assertTrue(diff.compareTo(tolerance) <= 0,
                () -> "x=" + x + " expected=" + expected + " actual=" + actual +
                        " diff=" + diff + " tolerance=" + tolerance);

        assertTrue(Cosine.getLastTermsCount() > 0, "Used terms count should be >
0");
        assertTrue(Cosine.getLastTermsCount() <= 1000, "Used terms count should not
exceed maxTerms");
    }

    static Stream<Arguments> valuesProvider() {

        BigDecimal PI = new BigDecimal(Math.PI, MC);

        return Stream.of(

                // 0
                Arguments.of(BigDecimal.ZERO),

                // ±π
                Arguments.of(PI),
                Arguments.of(PI.negate()),

                // ±π/2
                Arguments.of(PI.divide(new BigDecimal("2"), MC)),
                Arguments.of(PI.divide(new BigDecimal("2"), MC).negate()),

                // ±π/4
                Arguments.of(PI.divide(new BigDecimal("4"), MC)),
```

```java
                Arguments.of(PI.divide(new BigDecimal("4"), MC).negate()),

                // ±π/3
                Arguments.of(PI.divide(new BigDecimal("3"), MC)),
                Arguments.of(PI.divide(new BigDecimal("3"), MC).negate()),

                // π/6
                Arguments.of(PI.divide(new BigDecimal("6"), MC)),

                // 3π/4
                Arguments.of(PI.multiply(new BigDecimal("3"), MC)
                               .divide(new BigDecimal("4"), MC)),

                // 53π/180
                Arguments.of(PI.multiply(new BigDecimal("53"), MC)
                               .divide(new BigDecimal("180"), MC)),

                // 190π/180
                Arguments.of(PI.multiply(new BigDecimal("190"), MC)
                               .divide(new BigDecimal("180"), MC)),

                // -261π/180
                Arguments.of(PI.multiply(new BigDecimal("-261"), MC)
                               .divide(new BigDecimal("180"), MC)),

                // ±10⁻¹⁰
                Arguments.of(new BigDecimal("1e-10")),
                Arguments.of(new BigDecimal("-1e-10")),

                // ±10⁶
                Arguments.of(new BigDecimal("1e6")),
                Arguments.of(new BigDecimal("-1e6"))
        );
    }
}
```

Результаты тестирования

```
-------------------------------------------------------
 T E S T S
-------------------------------------------------------
Running org.example.math.CosineParameterizedTest
Tests run: 18, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.114 s - in org.example.math.CosineParameterizedTest

Results:

Tests run: 18, Failures: 0, Errors: 0, Skipped: 0
```

Element	Class, %	Method, %	Line, %	Branch, %
org.example	90% (9/10)	89% (53/59)	85% (223/261)	78% (110/140)
domain	85% (6/7)	87% (34/39)	78% (73/93)	94% (36/38)
heap	100% (2/2)	93% (15/16)	86% (118/136)	69% (61/88)
math	100% (1/1)	100% (4/4)	100% (32/32)	92% (13/14)
Cosine	100% (1/1)	100% (4/4)	100% (32/32)	92% (13/14)

Функция cos(x) корректно обрабатывает все протестированные случаи, включая граничные точки периода и большие по модулю значения. Все результаты находятся в пределах заданной точности и совпадают с эталонной реализацией Math.cos.

Реализация правильно выполняет редукцию аргумента по модулю (2π), корректно использует симметрию функции, а также корректно изменяет знак результата при отражении.

В целом тестирование подтверждает численную устойчивость алгоритма, корректность редукции аргумента и соответствие результатов эталонной реализации в рамках заданных допусков.

Часть 2

Реализация по варианту:
[ITMO/ТПО/Лаб1/Code/TPOLAB1/src/main/java/org/example/heap/Fibonacci.java at master · AlexandroLe/ITMO]

Фибоначчиева куча (англ. Fibonacci heap) – структура данных, отвечающая интерфейсу priority queue. Имеет меньшую амортизированную сложность, чем такие приоритетные очереди как биномиальная куча и двоичная куча.

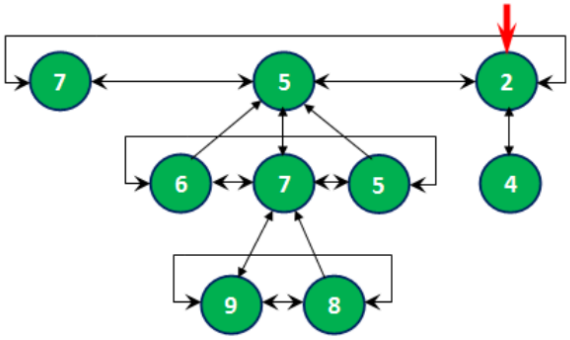


Рисунок 1. Схема фибоначчиевой кучи

Фибоначчиева куча представляет собой набор фибоначчиевых деревьев.

Фибоначчиево дерево – это k-ичное дерево, для каждого элемента которого выполняется правило: «дочерний элемент не превышает своего родителя». Корни фибоначчиевых деревьев хранятся в виде кольцевого списка. Каждый узел фибоначчиева дерева, помимо указателей на левого, правого брата, на родителя и на одного из своих сыновей содержит информацию о количестве дочерних узлов.

Основные операции:

- Поиск минимального элемента (*getMinimum*)
- Добавление нового элемента (*insert*)
- Объединение двух фибоначчиевых куч (*merge*)
- Удаление минимального элемента (*removeMinimum*)
- Изменение (приоритета) элемента (*decreaseKey*)

Целью модульного тестирования будет покрытие характерных точек внутри алгоритма и проверка в них корректности свойств структуры данных.

План тестирования

insert / вставка

1. **Вставка в пустую кучу**
   - Проверка min и структуры (root list корректно связан).
2. **Вставка нескольких элементов в непустую кучу**
   - Проверка min после каждой вставки.
   - Проверка структуры root list и дочерних списков.
3. **Вставка двух одинаковых минимумов**
   - Проверка min и связей в root list.
4. **Вставка нулевого значения**
   - Проверка корректности min и структуры.
5. **Вставка в обратном порядке**
   - Проверка min и структуры root list.
6. **Вставка в случайном порядке**
   - Проверка min и структуры root list.

getMinimum / min()

- Проверка корректного возвращения min после вставок (покрывается тестами insert).
- Проверка на пустой куче → min = null.

merge / объединение двух куч

1. **Слияние с пустой кучей**
   - Результат = исходная куча, min проверяется внутри теста.
2. **Слияние двух пустых куч**
   - min = null, куча пуста.
3. **Слияние двух непустых куч**
   - min = минимальный элемент среди обеих куч.
   - Полная проверка состояния root list и связей.

removeMinimum / удаление минимального элемента

1. **Удаление минимального на пустой куче**
   - Должно возвращать null, не выбрасывать исключений.
2. **Удаление минимального на куче с одним элементом**
   - Куча становится пустой, min = null.
3. **Удаление минимального на куче с несколькими элементами**
   - min обновляется корректно.
   - Проверка структуры root list и дочерних списков.
   - Проверка последовательного удаления нескольких элементов.

decreaseKey / уменьшение ключа

1. **Уменьшение ключа так, что элемент становится новым min**
   - Проверка min и root list после операции.
2. **Уменьшение ключа, не меняющее min**
   - Проверка min и структуры.

3. **Каскадный cut**
    - Проверка родительских отметок и root list после нескольких decreaseKey.
4. **Попытка увеличить ключ**
    - Должно выбрасывать `IllegalArgumentException`.
5. **Уменьшение ключа до нуля**
    - Проверка корректного поведения.

delete / удаление произвольного элемента

- Удаление элемента с/без дочерних узлов.
- Проверка min после удаления.
- Проверка структуры root list и дочерних списков.

removeFromRootList

- Удаление корня из root list на куче с несколькими элементами.
- Удаление единственного элемента в root list.
- Проверка корректности left/right ссылок после удаления.

Нагрузочные тесты

1. **Массовая вставка элементов**
    - Проверка min после всех вставок.
2. **Массовое removeMin**
    - Проверка последовательного удаления всех элементов и корректного обновления min.
3. **Массовое decreaseKey**
    - Применяем к случайным/всех элементам, проверяем min и root list после операций.

Тесты для фибоначчиевой кучи

```java
package org.example.heap;

import org.junit.jupiter.api.Test;
import java.util.*;
import java.util.stream.IntStream;
import static org.junit.jupiter.api.Assertions.*;

public class FibonacciTest {

    @Test
    void testInsertIntoEmpty() {
        Fibonacci heap = new Fibonacci();
        Fibonacci.Node n = heap.insert(42);
        assertEquals(42, heap.min().key);
        verifyStructure(heap);
    }
```

```java
@Test
void testInsertMultipleElements() {
    Fibonacci heap = new Fibonacci();
    int[] values = {5, 3, 7, 2, 8};
    for (int v : values) {
        heap.insert(v);
        verifyStructure(heap);
    }
    assertEquals(2, heap.min().key);
}

@Test
void testInsertDuplicateMin() {
    Fibonacci heap = new Fibonacci();
    heap.insert(3);
    heap.insert(3);
    heap.insert(5);
    assertEquals(3, heap.min().key);
    verifyStructure(heap);
}

@Test
void testInsertZeroValue() {
    Fibonacci heap = new Fibonacci();
    heap.insert(0);
    assertEquals(0, heap.min().key);
    verifyStructure(heap);
}

@Test
void testReversedInsert() {
    Fibonacci heap = new Fibonacci();
    for (int i = 5; i >= 1; i--) heap.insert(i);
    assertEquals(1, heap.min().key);
    verifyStructure(heap);
}

@Test
void testShuffledInsert() {
    Fibonacci heap = new Fibonacci();
    List<Integer> nums = Arrays.asList(3, 7, 1, 5, 9);
    Collections.shuffle(nums);
    for (int n : nums) heap.insert(n);
    assertEquals(1, heap.min().key);
    verifyStructure(heap);
}

@Test
void testMinOnEmptyHeap() {
    Fibonacci heap = new Fibonacci();
```

```java
        assertNull(heap.min());
    }

    @Test
    void testMergeWithEmpty() {
        Fibonacci h1 = new Fibonacci();
        Fibonacci h2 = new Fibonacci();
        h1.insert(5);
        h1.merge(h2);
        assertEquals(5, h1.min().key);
        verifyStructure(h1);
    }

    @Test
    void testMergeTwoEmpty() {
        Fibonacci h1 = new Fibonacci();
        Fibonacci h2 = new Fibonacci();
        h1.merge(h2);
        assertNull(h1.min());
        assertTrue(h1.isEmpty());
    }

    @Test
    void testMergeNonEmptyHeaps() {
        Fibonacci h1 = new Fibonacci();
        Fibonacci h2 = new Fibonacci();
        h1.insert(10); h1.insert(5);
        h2.insert(7); h2.insert(2);
        h1.merge(h2);
        assertEquals(2, h1.min().key);
        verifyStructure(h1);
    }

    @Test
    void testRemoveMinEmpty() {
        Fibonacci heap = new Fibonacci();
        assertNull(heap.removeMin());
    }

    @Test
    void testRemoveMinSingle() {
        Fibonacci heap = new Fibonacci();
        heap.insert(42);
        Object min = heap.removeMin();
        assertEquals(42, min);
        assertNull(heap.min());
    }

    @Test
    void testRemoveMinMultiple() {
        Fibonacci heap = new Fibonacci();
```

```java
        int[] values = {4, 1, 3};
        for (int v : values) heap.insert(v);

        Object min = heap.removeMin();
        assertEquals(1, min);
        assertEquals(3, heap.min().key);
        verifyStructure(heap);

        min = heap.removeMin();
        assertEquals(3, min);
        assertEquals(4, heap.min().key);
        verifyStructure(heap);

        min = heap.removeMin();
        assertEquals(4, min);
        assertNull(heap.min());
    }

    @Test
    void testDecreaseKeyNewMin() {
        Fibonacci heap = new Fibonacci();
        Fibonacci.Node n1 = heap.insert(10);
        heap.insert(20);
        heap.decreaseKey(n1, 5);
        assertEquals(5, heap.min().key);
        verifyStructure(heap);
    }

    @Test
    void testDecreaseKeyNoMinChange() {
        Fibonacci heap = new Fibonacci();
        Fibonacci.Node n1 = heap.insert(10);
        heap.insert(5);
        heap.decreaseKey(n1, 8);
        assertEquals(5, heap.min().key);
        verifyStructure(heap);
    }

    @Test
    void testDecreaseKeyCascade() {
        Fibonacci heap = new Fibonacci();
        Fibonacci.Node n1 = heap.insert(10);
        Fibonacci.Node n2 = heap.insert(20);
        Fibonacci.Node n3 = heap.insert(30);
        heap.removeMin();
        heap.decreaseKey(n3, 5);
        assertEquals(5, heap.min().key);
        verifyStructure(heap);
    }

    @Test
```

```java
    void testDecreaseKeyIllegal() {
        Fibonacci heap = new Fibonacci();
        Fibonacci.Node n1 = heap.insert(10);
        assertThrows(IllegalArgumentException.class, () -> heap.decreaseKey(n1,
15));
    }

    @Test
    void testDecreaseKeyToZero() {
        Fibonacci heap = new Fibonacci();
        Fibonacci.Node n1 = heap.insert(10);
        heap.decreaseKey(n1, 0);
        assertEquals(0, heap.min().key);
        verifyStructure(heap);
    }

    @Test
    void testDeleteElement() {
        Fibonacci heap = new Fibonacci();
        Fibonacci.Node n = heap.insert(10);
        heap.delete(n);
        assertNull(heap.min());
        verifyStructure(heap);
    }

    @Test
    void testRemoveFromRootListMultiple() {
        Fibonacci heap = new Fibonacci();
        Fibonacci.Node n1 = heap.insert(10);
        Fibonacci.Node n2 = heap.insert(5);
        heap.removeFromRootList(n1);
        assertEquals(5, heap.min().key);
        verifyStructure(heap);
    }

    @Test
    void testRemoveFromRootListSingle() {
        Fibonacci heap = new Fibonacci();
        Fibonacci.Node n1 = heap.insert(10);
        heap.removeFromRootList(n1);
        assertNull(heap.min());
    }

    @Test
    void stressInsert() {
        Fibonacci heap = new Fibonacci();
        IntStream.rangeClosed(1, 100000).forEach(heap::insert);
        assertEquals(1, heap.min().key);
    }

    @Test
```

```java
    void stressRemoveMin() {
        Fibonacci heap = new Fibonacci();
        IntStream.rangeClosed(1, 100000).forEach(heap::insert);
        IntStream.rangeClosed(1, 100000).forEach(i -> {
            Fibonacci.Node currentMin = heap.min();
            if (currentMin != null) assertEquals(i, currentMin.key);
            heap.removeMin();
        });
        assertNull(heap.min());
    }

    @Test
    void stressDecreaseKey() {
        Fibonacci heap = new Fibonacci();
        List<Fibonacci.Node> nodes = new ArrayList<>();
        IntStream.rangeClosed(1, 10000).forEach(i -> nodes.add(heap.insert(i +
1000)));
        Collections.shuffle(nodes);
        for (Fibonacci.Node n : nodes) heap.decreaseKey(n, 0);
        assertEquals(0, heap.min().key);
        verifyStructure(heap);
    }
}
```

Результаты тестирования

```
-------------------------------------------------------
 T E S T S
-------------------------------------------------------
Running org.example.heap.FibonacciTest
Tests run: 24, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.148 s - in org.example.heap.FibonacciTest

Results:

Tests run: 24, Failures: 0, Errors: 0, Skipped: 0
```

Element	Class, %	Method, %	Line, %	Branch, %
org.example	90% (9/10)	89% (53/59)	85% (223/261)	78% (110/140)
domain	85% (6/7)	87% (34/39)	78% (73/93)	94% (36/38)
heap	100% (2/2)	93% (15/16)	86% (118/136)	69% (61/88)
Fibonacci	100% (2/2)	93% (15/16)	86% (118/136)	69% (61/88)
math	100% (1/1)	100% (4/4)	100% (32/32)	92% (13/14)

По результатам тестирования реализация Фибоначчиевой кучи корректно обрабатывает все предусмотренные сценарии, включая граничные случаи и работу с большими объёмами данных. В тестах сохраняются свойства кучи, а извлечение элементов происходит в корректном порядке.

Структура корректно функционирует после различных комбинаций операций, включая вставку, удаление минимума, объединение куч и уменьшение ключа. Поведение на пустой куче и при наличии одного элемента соответствует ожидаемому.

Проверки с большим количеством элементов подтверждают устойчивость реализации и отсутствие деградации производительности в рамках протестированных сценариев.

Таким образом, тестирование подтверждает корректность и стабильность реализации в пределах охваченных тестами случаев.

Часть 3

Предметная область: после недолгой поездки на аэромобиле Артур и старый магратеянин оказались у какой-то двери. Они вышли из машины и прошли в приемную, уставленную стеклянными столиками и наградами из прозрачного пластика. Почти сразу же над дверью в другом конце комнаты загорелся свет, и они вошли в нее.

Доменная модель для заданного текста:
[ITMO/ТПО/Лаб1/Code/TPOLAB1/src/main/java/org/example/domain at master · AlexandroLe/ITMO]

Описание классов и их методов

1. Aeromobile - аэромобиль, средство перемещения персонажей

Aeromobile(String id) - создание аэромобиля с уникальным идентификатором.
getPassengers() - получение списка текущих пассажиров (только для чтения).
boardAllWithMarkers(List<Person>) - посадка группы персонажей в аэромобиль с удалением их из предыдущей локации.
approachDoor(Door) - приближение к двери (инициирует реакцию двери, например включение света).
disembarkAllWithMarkers(Location) - высадка всех пассажиров в указанную локацию.
toString() - строковое представление объекта.

2. Door - дверь между двумя локациями

Door(String name, Location a, Location b, Light light) - создание двери, соединяющей две локации, с источником света.
getName() - получение имени двери.
getLight() - получение связанного источника света.
lock() - блокировка двери.
unlock() - разблокировка двери.
isLocked() - проверка состояния блокировки.
otherSide(Location) - определение противоположной стороны двери относительно текущей локации.
approachWithMarkers() - реакция двери на приближение (включение света).
passThrough(Location) - переход через дверь при отсутствии блокировки.
toString() - строковое представление двери.

3. Item - предмет в локации

Item(String name, String material) - создание предмета с названием и материалом.

4. Light - источник света

isOn() - проверка состояния света (включён/выключен).
turnOn() - включение света.
turnOff() - выключение света.

5. Location - место действия (пространство)

Location(String name, String description) - создание локации с названием и описанием.
getName() - получение названия локации.
getDescription() - получение описания локации.
addOccupant(Person) - добавление персонажа в локацию.
removeOccupant(Person) - удаление персонажа из локации.
getOccupants() - получение списка находящихся в локации персонажей.
getItems() - получение списка предметов в локации.
getDoors() - получение списка дверей, связанных с локацией.
addItem(Item) - добавление предмета в локацию.
addDoor(Door) - добавление двери к локации.
toString() - строковое представление локации.

6. Person - персонаж доменной модели

Person(String name) - создание персонажа без начальной локации.
Person(String name, Location initial) - создание персонажа с размещением в начальной локации.
getName() - получение имени персонажа.
getLocation() - получение текущей локации персонажа.
moveTo(Location) - перемещение персонажа в другую локацию с обновлением состояния обеих локаций.
inspectWithMarkers() - действие осмотра текущей локации.
toString() - строковое представление персонажа.

7. Main - демонстрация сценария

main(String[] args) - моделирование последовательности событий из текста: прибытие к двери, вход в приёмную, включение света и изменение состояний объектов.

Тесты доменной модели

Тесты всех классов: [ITMO/ТПО/Лаб1/Code/TPOLAB1/src/test/java/org/example/domain at master · AlexandroLe/ITMO]

Результаты тестирования

Element	Class, %	Method, %	Line, %	Branch, %
org.example	90% (9/10)	89% (53/59)	85% (223/261)	78% (110/140)
domain	85% (6/7)	87% (34/39)	78% (73/93)	94% (36/38)
Aeromobile	100% (1/1)	83% (5/6)	94% (16/17)	100% (8/8)
Door	100% (1/1)	90% (9/10)	95% (22/23)	100% (10/10)
Item	100% (1/1)	100% (1/1)	100% (1/1)	100% (0/0)
Light	100% (1/1)	100% (3/3)	100% (3/3)	100% (0/0)
Location	100% (1/1)	90% (10/11)	93% (15/16)	100% (10/10)
Main	0% (0/1)	0% (0/1)	0% (0/16)	0% (0/2)
Person	100% (1/1)	85% (6/7)	94% (16/17)	100% (8/8)

Тесты, соответствующие всем классам, корректно проходят. Поведение объектов и их методов соответствует ожидаемому.

Выводы

В ходе лабораторной работы мы реализовали и протестировали математическую функцию, структуру данных и простую доменную модель. Это позволило нам на практике применить разные подходы к тестированию.

При работе с функцией косинуса мы уделили внимание точности вычислений и проверке граничных значений. В задаче с кучей мы сосредоточились на проверке корректности операций и их поведения при разных входных данных, включая большие объёмы. В доменной модели основной акцент был сделан на корректности взаимодействия объектов и изменении их состояний.

В результате мы закрепили навыки написания модульных тестов, научились продумывать тестовые сценарии для обычных и граничных случаев и получили больше опыта в проверке как вычислительной логики, так и поведения объектов в системе.